

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO5-AT1 Pseudocode Design

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5-AT1 Pseudocode Design	Assessment:	CO5-AT1 Pseudocode Design
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)		
Student Name:	B Tharun Kumar	Reg. No.:	192424356
Section / Batch:	AI Data Science	Date:	25-08-2026

Code of Conduct

I B Tharun Kumar/192424356 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *B Tharun Kumar*

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Pseudocode Assignment

1. CART Classification Algorithm using Gini Impurity

Problem: Design pseudocode for the CART classification algorithm to select the best binary split using Gini impurity and construct a classification tree.

Pseudocode:

BEGIN CART_Classification

FUNCTION GiniImpurity(dataset):

 classes = distinct class labels in dataset

 total = number of records in dataset

 gini = 1

 FOR each class c in classes:

 p_c = count(records with label c) / total

 gini = gini - (p_c * p_c)

 RETURN gini

FUNCTION GiniSplit(left_subset, right_subset):

 n_left = size(left_subset)

 n_right = size(right_subset)

 n_total = n_left + n_right

 weighted_gini = (n_left/n_total)*GiniImpurity(left_subset) +
 (n_right/n_total)*GiniImpurity(right_subset)

 RETURN weighted_gini

FUNCTION FindBestSplit(dataset, features):

 best_gini = INFINITY

 best_feature = NULL

 best_threshold = NULL

 FOR each feature F in features:

 FOR each unique value V of feature F in dataset:

 left_subset = records where $F \leq V$

 right_subset = records where $F > V$

 IF left_subset is empty OR right_subset is empty:

 CONTINUE

 current_gini = GiniSplit(left_subset, right_subset)

```
IF current_gini < best_gini:  
    best_gini = current_gini  
    best_feature = F  
    best_threshold = V
```

```
RETURN best_feature, best_threshold, best_gini
```

```
FUNCTION BuildTree(dataset, depth):
```

```
    IF all records in dataset belong to same class:
```

```
        RETURN LeafNode(class label)
```

```
    IF depth == max_depth OR size(dataset) < min_samples_split:
```

```
        RETURN LeafNode(majority class in dataset)
```

```
    feature, threshold, gini = FindBestSplit(dataset, features)
```

```
    IF feature is NULL:
```

```
        RETURN LeafNode(majority class in dataset)
```

```
    left_data = records where feature <= threshold
```

```
    right_data = records where feature > threshold
```

```
    left_child = BuildTree(left_data, depth + 1)
```

```
    right_child = BuildTree(right_data, depth + 1)
```

```
    RETURN DecisionNode(feature, threshold, left_child, right_child)
```

```
MAIN:
```

```
    root = BuildTree(training_dataset, depth = 0)
```

```
    RETURN root
```

```
END CART_Classification
```

2. Decision Tree for Customer Purchase Prediction

Problem: Design pseudocode to predict whether a customer will purchase a product using a Decision Tree based on age, income, previous purchases, and browsing history.

Pseudocode:

BEGIN Customer_Purchase_Prediction

INPUT: dataset with features [Age, Income, PreviousPurchases, BrowsingHistory] and target label Purchased (Yes/No)

FUNCTION TrainDecisionTree(dataset, features, depth):

 IF dataset is empty:

 RETURN NULL

 IF all records have same Purchased value:

 RETURN LeafNode(Purchased value)

 IF depth == max_depth OR features is empty:

 RETURN LeafNode(majority class of Purchased)

 best_feature = feature in features that gives maximum
 Information Gain (or minimum Gini Impurity)
 when splitting dataset

 tree = DecisionNode(best_feature)

 FOR each possible value/range V of best_feature:

 subset = records in dataset where best_feature == V

 IF subset is empty:

 tree.addBranch(V, LeafNode(majority class of dataset))

 ELSE:

 remaining_features = features - {best_feature}

 child = TrainDecisionTree(subset, remaining_features, depth+1)

 tree.addBranch(V, child)

 RETURN tree

FUNCTION Predict(tree, customer):

 IF tree is LeafNode:

 RETURN tree.label // "Yes" or "No"

 feature_value = customer[tree.feature]

 next_node = tree.branch(feature_value)

 RETURN Predict(next_node, customer)

MAIN:

```
features = [Age, Income, PreviousPurchases, BrowsingHistory]  
model = TrainDecisionTree(training_dataset, features, depth = 0)
```

```
new_customer = {Age: 28, Income: 45000,  
                PreviousPurchases: 3, BrowsingHistory: High}
```

```
result = Predict(model, new_customer)
```

```
PRINT "Will customer purchase?", result
```

END Customer_Purchase_Prediction

3 Regression Analysis Workflow

Problem: Design pseudocode for a regression analysis workflow that loads a dataset, identifies independent and dependent variables, trains a regression model, and evaluates its predictions.

Pseudocode:

BEGIN Regression_Analysis_Workflow

STEP 1: Load Dataset

dataset = READ_DATA(file_path)

STEP 2: Identify Variables

independent_variables (X) = all predictor columns in dataset

dependent_variable (Y) = target/outcome column in dataset

STEP 3: Preprocess Data

Handle missing values in dataset

Encode categorical variables if any

Normalize/Scale numerical features

STEP 4: Split Dataset

(X_train, X_test, Y_train, Y_test) = SPLIT(X, Y,
train_ratio = 0.8)

STEP 5: Train Regression Model

model = RegressionModel()

model.fit(X_train, Y_train)

STEP 6: Make Predictions

Y_predicted = model.predict(X_test)

STEP 7: Evaluate Model

MAE = MeanAbsoluteError(Y_test, Y_predicted)

MSE = MeanSquaredError(Y_test, Y_predicted)

RMSE = SQRT(MSE)

R2 = RSquared(Y_test, Y_predicted)

PRINT "MAE:", MAE

PRINT "MSE:", MSE

PRINT "RMSE:", RMSE

PRINT "R-Squared:", R2

STEP 8: Output Final Model

 RETURN trained model and evaluation metrics

END Regression_Analysis_Workflow

4. Simple Linear Regression (House Price from Area)

Problem: Design pseudocode to implement simple linear regression for predicting house price from house area using the least-squares approach.

Pseudocode:

BEGIN Simple_Linear_Regression

INPUT: dataset with House_Area (X) and House_Price (Y)

STEP 1: Load Data

 X = list of house areas

 Y = list of house prices

 n = number of data points

STEP 2: Calculate Means

 mean_X = SUM(X) / n

 mean_Y = SUM(Y) / n

STEP 3: Calculate Slope (m) using Least Squares

 numerator = 0

 denominator = 0

 FOR i = 1 to n:

 numerator = numerator + (X[i]-mean_X)*(Y[i]-mean_Y)

 denominator = denominator + (X[i]-mean_X)^2

 m = numerator / denominator

STEP 4: Calculate Intercept (b)

 b = mean_Y - (m * mean_X)

STEP 5: Form Regression Equation

 Price = m * Area + b

STEP 6: Predict Price for New Area

 FUNCTION Predict(area):

 RETURN m * area + b

 new_area = 1500

 predicted_price = Predict(new_area)

 PRINT "Predicted Price:", predicted_price

STEP 7: Evaluate Model

 Y_predicted = [Predict(X[i]) for i = 1 to n]

```
RMSE = SQRT(SUM((Y[i]-Y_predicted[i])^2)/n)
```

```
R2 = RSquared(Y, Y_predicted)
```

```
PRINT "RMSE:", RMSE, "R-Squared:", R2
```

```
END Simple_Linear_Regression
```

5. Multiple Linear Regression (Student Final Mark)

Problem: Design pseudocode to implement multiple linear regression for predicting a student's final mark using attendance, internal marks, assignment marks, and study hours.

Pseudocode:

BEGIN Multiple_Linear_Regression

INPUT: dataset with independent variables

 X1 = Attendance

 X2 = Internal_Marks

 X3 = Assignment_Marks

 X4 = Study_Hours

 dependent variable Y = Final_Mark

STEP 1: Load and Prepare Data

 dataset = READ_DATA(file_path)

 X = matrix of [X1, X2, X3, X4] for all students

 Y = vector of Final_Mark for all students

 n = number of students (rows)

STEP 2: Add Bias/Intercept Term

 Add a column of 1's to X (for intercept term b0)

 X = [1, X1, X2, X3, X4]

STEP 3: Split Dataset

 (X_train, X_test, Y_train, Y_test) = SPLIT(X, Y, 0.8)

STEP 4: Estimate Coefficients using Least Squares

 // Using Normal Equation: $B = (X^T * X)^{-1} * X^T * Y$

 B = MATRIX_INVERSE(TRANSPPOSE(X_train) * X_train)

 * TRANSPPOSE(X_train) * Y_train

 // B = [b0, b1, b2, b3, b4]

STEP 5: Form Regression Equation

 Final_Mark = b0 + b1*Attendance + b2*Internal_Marks
 + b3*Assignment_Marks + b4*Study_Hours

STEP 6: Predict Final Marks

 FUNCTION Predict(attendance, internal, assignment, study_hrs):

 RETURN b0 + b1*attendance + b2*internal

+ b3*assignment + b4*study_hrs

Y_predicted = Predict(X_test values)

STEP 7: Evaluate Model

MAE = MeanAbsoluteError(Y_test, Y_predicted)

MSE = MeanSquaredError(Y_test, Y_predicted)

RMSE = SQRT(MSE)

R2 = RSquared(Y_test, Y_predicted)

PRINT "MAE:", MAE, "RMSE:", RMSE, "R-Squared:", R2

STEP 8: Output Final Model

RETURN coefficients B and evaluation metrics

END Multiple_Linear_Regression